WPT Day 14 Morning — Organising a React CRUD UI and Connecting It to a Real Backend

> **Instructor:** Abhishek Purohit · **Date:** Tue, 3 Nov 2026 (morning) · **Scope:** Split a single-file React screen into an `AppLayout` with typed children, introduce nested routes with `<Outlet />`, move logic into custom hooks, memoise components and callbacks, render database rows in an MUI table, then fix the CORS block and replace `fetch` with Axios.

Table of Contents

1. Where We Left Off
2. Lifting State Up
3. Typed Props for the Layout Children
4. Nested Routes and the Outlet
5. Navigating from the Side Nav
6. Extracting the App Router
7. Custom Hooks and Component Folders
8. Memoization with memo, useMemo and useCallback
9. Building the Product Table
10. Fetching Products from Our Backend
11. CORS: Why the Request Was Blocked
12. Axios and a Central API Client
13. Combined Worked Example
14. Quick Revision Checklist
15. Practice Exercises
16. Practice Answers

1. Where We Left Off

Problem. The previous session had built the application top bar (navbar) and the drawer menu (side nav) **inside the same `app.tsx`**. That file worked, but it was mixing several components in one place. The lecture's rule is blunt: *we should always avoid that approach*.

The first refactor had already moved the two visual pieces into their own files:

```
src/
├── App.tsx
└── components/
    ├── application-topbar.tsx
    └── sidenav.tsx
```

At the start of this session the project was copied into a new folder, `3rd November morning`, with `node_modules` deliberately left behind, and dependencies were restored with `npm install` (or

`npm i`). Copying without `node_modules` is normal: the folder is huge, and `package.json` + `package-lock.json` already describe everything needed to rebuild it.

The new problem. Splitting the UI exposed a dependency that had been invisible while everything lived in one file:

- `sidenav.tsx` owned the `openDrawer` state and the functions that open and close the drawer.
- `application-topbar.tsx` also needed one of those functions — the menu icon in the top bar must open the drawer.
- The two components are **siblings**. Neither is the parent of the other, so one cannot simply read the other's state or call its functions.

This is the central React data-flow rule: **props flow down, events flow up**. There is no sideways channel between siblings. The fix is not a new library; it is to move the shared data to a common parent.

```
Broken shape (siblings cannot share):

  App
   ├── ApplicationTopBar   ✗ needs handleOpenDrawer
   └── SideNav             owns openDrawer + handlers

Fixed shape (common parent owns the shared state):

  App
   └── AppLayout                    owns openDrawer + handlers
        ├── ApplicationTopBar      ← menuIconClickHandler
        └── SideNav                ← openDrawer, onDrawerClose
```

> **Exam:** State that two sibling components both need must live in their closest common parent. The parent passes values/functions down as props; children report events by calling those functions.

2. Lifting State Up

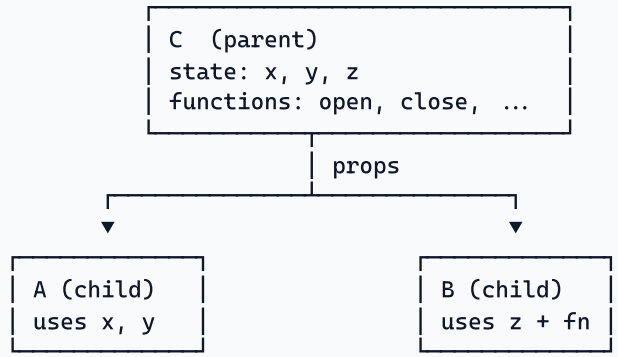
2.1 The A/B/C thought experiment

The lecture explains the pattern with three abstract components:

1. Component **A** owns state `x`, `y`, `z` and several functions.
2. Component **B** needs that same state and those same functions.
3. A and B have **no parent–child relationship** — they are adjacent/sibling components.
4. Therefore a third component, **C**, is created as their common parent.

5. The state and functions move **out of A and into C**.
6. C renders A and B, passing the state and functions down as props.
7. Everywhere the application used to render `<A />` and `<B />`, it now renders only `<C />`, because A and B are rendered inside C.

The move is called **lifting state up**: the state travels one level higher in the component tree so that more than one child can reach it.



2.2 Applying it to the real code: `AppLayout`

For this application, the common parent is not an abstract C — it is the **layout** of the whole product. The top bar and drawer must appear on *every* page: home, about, contact, services, dashboard, product list, and so on. That is exactly what a layout is: the reusable frame shared by all screens.

So the parent component is named `AppLayout` (the lecture also accepts `Layout` or `AppLayout`). It:

- owns `openDrawer` and the open/close handlers;
- renders `ApplicationTopBar` and `SideNav`;
- passes only what each child needs.

The drawer **menu items** are *not* lifted. They belong exclusively to the side nav, so they stay in the side nav (and later in its hook). Lifting is for genuinely shared state, not for "everything in one place".

```tsx
// src/components/app-layout/AppLayout.tsx
import { memo } from 'react';
import { Box, Toolbar } from '@mui/material';
import { Outlet } from 'react-router-dom';
import ApplicationTopBar from '../application-topbar/ApplicationTopBar';
import SideNav from '../sidenav/SideNav';
import { useAppLayout } from './useAppLayout';
```

```
export const AppLayout = memo(() => {
    const { openDrawer, handleOpenDrawer, handleCloseDrawer } = useAppLayout();

    return (
        <>
            <ApplicationTopBar menuIconClickHandler={handleOpenDrawer} />
            <SideNav openDrawer={openDrawer} onDrawerClose={handleCloseDrawer} />
            <Box component="main" sx={{ flexGrow: 1, p: 4 }}>
                <Toolbar />
                <Outlet />
            </Box>
        </>
    );
});
```

`useAppLayout` is introduced formally in section 7; for now read it as "the place where the lifted state lives".

2.3 Why an empty fragment `<> … </>` instead of a `div` or `Box`

A component can return only **one** value, but `AppLayout` must place the top bar, the drawer, and the main content next to each other. Something must wrap them.

Wrapper	What happens
`<div>`	Adds a real HTML element with browser defaults; can disturb the MUI layout hierarchy and bring unwanted styles
`<Box>` / `<section>`	Also a real element; MUI `Box` adds styling behaviour and unnecessary markup
`<> … </>` (`React.Fragment`)	A container for JSX with **no DOM node**; nothing is added to the HTML, so no layout or style conflict

The lecture calls this the **empty code fragment**: it is neither a `div`, nor a paragraph, nor any HTML element. It is just an empty container. Use it when you need a common parent but do not know (or do not want) a named element there.

> **Pitfall:** Two adjacent JSX elements cannot be returned side by side. Wrapping them in a `div` "works", but silently changes the DOM and can fight MUI's layout. Prefer a fragment when the wrapper has no semantic or styling job.

2.4 Cleaning `App.tsx`

Once `AppLayout` exists, `app.tsx` no longer contains the top bar, the drawer, their state, or their handlers. It renders the router and the layout, nothing else:

```tsx
// src/App.tsx — after the first refactor
import { BrowserRouter } from 'react-router-dom';
import { AppLayout } from './components/app-layout/AppLayout';

function App() {
    return (
        <BrowserRouter>
            <AppLayout />
        </BrowserRouter>
    );
}

export default App;
```

Running `npm run dev` showed the **same behaviour as before**: the top bar rendered and the menu icon still opened the drawer. The visual result was unchanged; only the ownership of the code had moved.

> **Exam:** Refactoring should be behaviour-preserving. If the UI changes after a pure reorganisation, something was moved incorrectly. "Same output, better structure" is the success criterion.

3. Typed Props for the Layout Children

3.1 What each child receives

The two children do not receive the same things. Deciding the prop list is part of the design:

Child	Needs	Prop name in the lecture	Prop type
`ApplicationTopBar`	A function to run when the menu icon is clicked	`menuIconClickHandler`	`() => void`
`SideNav`	Whether the drawer is open	`openDrawer`	`boolean`
`SideNav`	A function to close the drawer	`onDrawerClose`	`() => void`

The top bar does **not** need the state itself; it only needs the "open" action. The side nav needs the state because the MUI `Drawer` uses it to decide whether to show itself, and it needs the close action for its `onClose` and for closing after a menu click.

Naming matters. `handleOpenDrawer` describes *what the function does inside the parent*; `menuIconClickHandler` describes *the event it handles in the child*. The lecture deliberately renamed it at the boundary so the child's prop reads like an event.

3.2 The interfaces

```ts
// src/components/application-topbar/types.ts
export interface IApplicationTopBarProps {
    menuIconClickHandler: () => void;
}
```

```ts
// src/components/sidenav/types.ts
import type { JSX } from 'react';

export interface ISideNavProps {
    openDrawer: boolean;
    onDrawerClose: () => void;
}

export interface IDrawerMenuItem {
    text: string;
    icon: JSX.Element; // the lecture wrote React.JSX.Element
    path: string;
}
```

`() => void` is a **function type**: it takes no arguments and returns nothing. It is not a number, string, or boolean. Declaring it lets TypeScript reject a wrong value, such as passing a string to `menuIconClickHandler` or calling it with arguments it does not accept. The implementation (`() => setOpenDrawer(true)`) is compatible because its return value can be ignored.

`JSX.Element` is the type of "a rendered React element", which is what `<DashboardIcon />` produces. An icon is not a string or a number. The lecture wrote `React.JSX.Element`; importing the `JSX` namespace type gives the same meaning.

3.3 Wiring the props at the call site

```tsx
// inside AppLayout
<ApplicationTopBar menuIconClickHandler={handleOpenDrawer} />
<SideNav openDrawer={openDrawer} onDrawerClose={handleCloseDrawer} />
```

The direction of travel is always one way:

```
AppLayout (state + handlers)
   │
   ├── menuIconClickHandler ──────────────► ApplicationTopBar
   │                                           (click) ──► calls the function
   │
   ├── openDrawer ────────────────────────► SideNav (Drawer open={openDrawer})
   └── onDrawerClose ─────────────────────► SideNav (onClose / after navigation)
```

`handleOpenDrawer` lives in the parent, is passed down as a value, and is *called* in the child. The child never sets the parent's state directly; it calls the function it was given.

> **Exam:** A child cannot mutate a parent's state directly. It calls a callback prop, and the callback (defined in the parent, where the setter lives) performs the update.

4. Nested Routes and the Outlet

4.1 Creating pages

Pages are not reusable UI components, so they do not go in `components/`. They go in `src/pages/`, one folder per screen. A folder is used because a screen may later need its own hook and its own `types.ts`, exactly like a component.

```
src/pages/
├── dashboard/
│   ├── Dashboard.tsx
│   └── types.ts          (only if the page needs types)
├── products-list/
│   ├── ProductsList.tsx
│   ├── useProductsList.ts
│   └── types.ts
└── create-product/
    ├── CreateProduct.tsx
    └── types.ts
```

For now each page is deliberately tiny, just enough to prove routing works:

```tsx
// src/pages/dashboard/Dashboard.tsx
import { memo } from 'react';
import { Typography } from '@mui/material';

export const Dashboard = memo(() => (
    <Typography variant="h1" component="h1">
        Welcome to the product management app
```

```
    </Typography>
));
```

```
// src/pages/products-list/ProductsList.tsx (placeholder version)
import { memo } from 'react';
import { Typography } from '@mui/material';

export const ProductsList = memo(() => (
    <Typography variant="h1" component="h1">
        List of products
    </Typography>
));
```

```
// src/pages/create-product/CreateProduct.tsx (placeholder version)
import { memo } from 'react';
import { Typography } from '@mui/material';

export const CreateProduct = memo(() => (
    <Typography variant="h1" component="h1">
        Add a new product
    </Typography>
));
```

`Typography` with `variant="h1"` and `component="h1"` renders a real `<h1>` element with the theme's h1 typography. The `component` prop is what changes the underlying HTML tag.

4.2 First routing attempt — and why the layout looked wrong

The routes were declared in `App.tsx`:

```
<Routes>
    <Route path="/" element={<Dashboard />} />
    <Route path="/products-list" element={<ProductsList />} />
    <Route path="/add-product" element={<CreateProduct />} />
</Routes>
```

Typing the URLs worked: `/` showed the welcome message, `/products-list` showed "List of products", and `/add-product` showed "Add a new product". But the UI was **distorted**. The reason is visible in the JSX tree:

```
<BrowserRouter>
    <AppLayout />        ← one child of BrowserRouter
    <Routes>             ← sibling of AppLayout!
        <Route ... />
```

```
    </Routes>
</BrowserRouter>
```

`AppLayout` and the routed page are **siblings rendered at the same level**. The layout does not contain the page; it sits beside it. So the layout cannot decide where the page appears, and the page is not protected by the layout's structure.

The design rule from the lecture: a layout component carries the **sole responsibility** of deciding the entire frontend layout. It must therefore own the page as a child, not compete with it.

4.3 Solution 1: wrap every route element

The direct fix is to make each page a child of the layout:

```
<Routes>
    <Route
        path="/"
        element={
            <AppLayout>
                <Dashboard />
            </AppLayout>
        }
    />
    <Route
        path="/products-list"
        element={
            <AppLayout>
                <ProductsList />
            </AppLayout>
        }
    />
</Routes>
```

This works, but it repeats `<AppLayout>` for every route. In a real application with dozens or hundreds of screens, that is exactly the kind of repetition the session is trying to remove.

Approach	Correct nesting?	Repetition	Verdict
Layout and routes as siblings	No	None	Wrong layout
Wrap each route element in `<AppLayout>`	Yes	Layout repeated per route	Acceptable but not ideal
Nested route with parent `AppLayout`	Yes	Declared once	Lecture's solution

4.4 Solution 2: nested routes

A route can contain other routes. The parent route renders the layout once, and the child routes render into a placeholder inside that layout.

```
<Routes>
    <Route path="/" element={<AppLayout />}>
        <Route index element={<Dashboard />} />
        <Route path="products-list" element={<ProductsList />} />
        <Route path="add-product" element={<CreateProduct />} />
    </Route>
</Routes>
```

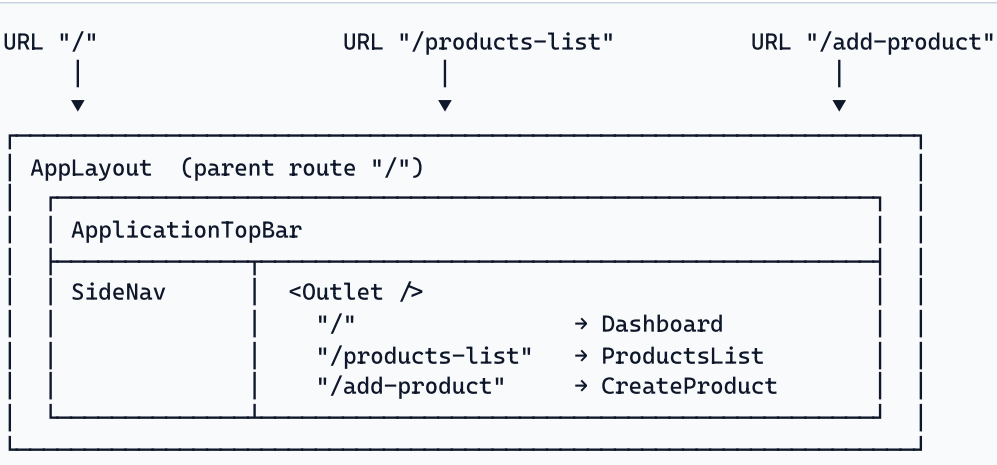
How the router reads this:

1. For **every** URL, the parent route `path="/"` matches first, so `AppLayout` renders.
2. The router then looks at the rest of the URL and matches a child route.
3. The matching child is rendered **where the parent says children go**.

That "where" is the `<Outlet />` component from `react-router-dom`:

```
<Box component="main" sx={{ flexGrow: 1, p: 4 }}>
    <Toolbar />
    <Outlet />
</Box>
```

`Outlet`'s own documentation description, quoted in the lecture, is: "*renders the matching child route of a parent route.*" It is a **placeholder**, not a hard-coded component.



Why `Outlet` and not a hard-coded `<Dashboard />`? Because at the moment `AppLayout` is written, **nobody can know which page is current**. That depends on the browser URL, which is

known only at runtime by the router. The layout cannot guess between dashboard, product list, create product, profile, or settings. `Outlet` is the dynamic slot the router fills.

The `index` route is the child that matches when the parent URL matches exactly (`/` with nothing after it). The child paths are written **relative** (`products-list` , not `/products-list`) so they are unambiguously nested under the parent.

> **Exam:** `<Outlet />` is the placeholder for a parent route's matched child. It lets one layout wrap many pages without repeating the layout in every `element={ ... }` .

4.5 Adjusting the content position

Once the page is genuinely inside the layout, it sat under the fixed top bar and needed spacing. The lecture used the MUI `sx` prop:

```
<Box component="main" sx={{ flexGrow: 1, p: 4 }}>
    <Toolbar />
    <Outlet />
</Box>
```

- `flexGrow: 1` lets the content area take the remaining width/space.
- `p: 4` gives uniform padding on all four sides (MUI spacing unit 4 = 32px by default).
- `<Toolbar />` is an **empty spacer** with the same height as the app bar. Because the `AppBar` is fixed, without this spacer the first content line hides underneath it.

The lecture notes that margin is an equally valid choice; `Toolbar` and margin are both acceptable ways to clear a fixed header.

5. Navigating from the Side Nav

Problem. Routing by typing a URL works, but users interact with the side nav. Each drawer item is a `ListItemButton` , not an `<a>` link, so clicking it does nothing until a click handler navigates.

5.1 The hook

`react-router-dom` provides `useNavigate` . It returns a **function**; calling that function changes the route programmatically.

```
import { useNavigate } from 'react-router-dom';

const navigate = useNavigate();
navigate('/products-list');
```

5.2 The hard-coding trap

The menu is rendered inside `drawerMenu.map( ... )`. One handler body serves every iteration, so it cannot hard-code `navigate('/')` for one item and `navigate('/products-list')` for another. Doing so would send every menu item to the same page.

The lecture's fix is to enrich each menu item with its own destination:

```
export interface IDrawerMenuItem {
    text: string;
    icon: JSX.Element;
    path: string; // where this item should navigate
}
```

```
const drawerMenu: IDrawerMenuItem[] = [
    { text: 'Dashboard', icon: <DashboardIcon />, path: '/' },
    { text: 'Products List', icon: <ListAltIcon />, path: '/products-list' },
    { text: 'Create Product', icon: <AddBoxIcon />, path: '/add-product' },
];
```

Then the click handler reads the path from the current iteration:

```
onClick={() => navigate(menuItem.path)}
```

Menu item	`path`	Route that opens
Dashboard	`/`	`Dashboard` (index route)
Products List	`/products-list`	`ProductsList`
Create Product	`/add-product`	`CreateProduct`

`path` is a string and must be added to the interface; until it is, TypeScript reports that the property does not exist.

5.3 The second bug: the drawer stayed open

After navigation worked, clicking a menu item changed the page but **the drawer remained open**. The user expects the drawer to close after choosing an item, especially on a narrow screen where it overlays the content.

The side nav already receives `onDrawerClose` from `AppLayout`, and that callback sets `openDrawer` to `false`. So the click handler must do two things, in order:

```
const handleMenuClick = (path: string) => {
    navigate(path);      // 1. go to the page
    onDrawerClose();     // 2. hide the drawer
};
```

The state itself lives in the parent hook; the child only calls the callback. Closing the drawer is therefore another example of "event up, state down".

> **Pitfall:** Calling `navigate` is not enough. The drawer's `open` state is independent of the URL, so navigation alone leaves it visible. Call the close callback after navigating.

6. Extracting the App Router

Problem. After adding pages, `App.tsx` contained every route. With hundreds of routes, that file becomes unmanageable, and `App` stops being "neat and clean".

The lecture extracts the routing into its own component, `AppRouter`. A common question is whether this belongs in `pages/` or `components/`. The answer: a router is itself a component (it returns JSX), so it goes in `components/`. The `components/` folder was never restricted to purely visual widgets; if desired it can later be split into `components/ui/` and `components/routing/`.

```
src/components/app-router/
└── AppRouter.tsx
```

```
// src/components/app-router/AppRouter.tsx
import { memo } from 'react';
import { Route, Routes } from 'react-router-dom';
import { AppLayout } from '../app-layout/AppLayout';
import { Dashboard } from '../../pages/dashboard/Dashboard';
import { ProductsList } from '../../pages/products-list/ProductsList';
import { CreateProduct } from '../../pages/create-product/CreateProduct';

export const AppRouter = memo(() => (
    <Routes>
        <Route path="/" element={<AppLayout />}>
            <Route index element={<Dashboard />} />
            <Route path="products-list" element={<ProductsList />} />
            <Route path="add-product" element={<CreateProduct />} />
        </Route>
    </Routes>
));
```

```tsx
// src/App.tsx — final shape
import { BrowserRouter } from 'react-router-dom';
import { AppRouter } from './components/app-router/AppRouter';

function App() {
    return (
        <BrowserRouter>
            <AppRouter />
        </BrowserRouter>
    );
}

export default App;
```

The responsibility chain is now explicit:

```
App
 └── AppRouter          decides which URL maps to which page
      └── AppLayout     decides where top bar, side nav and page sit
           ├── ApplicationTopBar
           ├── SideNav
           └── Outlet → Dashboard | ProductsList | CreateProduct
```

> **Exam:** `App` should only compose the outermost providers/router. Route tables belong in an `AppRouter` component; layout belongs in `AppLayout`; each screen belongs in `pages/`.

7. Custom Hooks and Component Folders

7.1 The problem: UI code and business logic mixed

`AppLayout.tsx` contained two different kinds of code:

- **UI code** — JSX that decides what is displayed.
- **Business logic** — `useState`, `handleOpenDrawer`, `handleCloseDrawer`.

The same was true of `sidenav.tsx`, which built the drawer menu array inline. Technically this works. The lecture's objection is maintenance: in a real component there may be 20–30 functions, each 10–20 lines, and a TSX file that holds all of them plus the markup becomes hard to read and change.

The rule: **UI stays in the `.tsx` component; business logic moves into a custom hook.**

7.2 Folder-per-component

Hooks were not dumped into one shared folder. For every component, create a folder containing:

```
components/<component-name>/
├── <ComponentName>.tsx   ← UI only
├── use<ComponentName>.ts  ← state, handlers, derived data
└── types.ts               ← interfaces used by the component and hook
```

Applied to this session:

```
src/components/
├── app-layout/
│   ├── AppLayout.tsx
│   ├── useAppLayout.ts
│   └── types.ts
├── application-topbar/
│   ├── ApplicationTopBar.tsx
│   └── types.ts
├── sidenav/
│   ├── SideNav.tsx
│   ├── useSideNav.tsx
│   └── types.ts
└── app-router/
    └── AppRouter.tsx
```

The hook file is usually `.ts`. It becomes `.tsx` when the hook itself returns JSX, for example menu items containing `<DashboardIcon />`. The lecture explicitly allows this: a hook that deals with components needs the TSX extension.

7.3 `useAppLayout`

```tsx
// src/components/app-layout/useAppLayout.ts
import { useCallback, useState } from 'react';

export function useAppLayout() {
    const [openDrawer, setOpenDrawer] = useState(false);

    const handleOpenDrawer = useCallback(() => {
        setOpenDrawer(true);
    }, []);

    const handleCloseDrawer = useCallback(() => {
        setOpenDrawer(false);
    }, []);

    return {
        openDrawer,
        setOpenDrawer,
        handleOpenDrawer,
        handleCloseDrawer,
    };
}
```

The component calls the hook once and destructures only what it needs:

```
const { openDrawer, handleOpenDrawer, handleCloseDrawer } = useAppLayout();
```

`setOpenDrawer` is returned too, in case another consumer needs the raw setter, but the layout does not use it. Return values are an **object**, not a positional tuple, so consumers can take only the fields they need and the order does not matter.

7.4 `useSideNav`

The side-nav hook is slightly different: it receives the component's props as an argument, because its click handler must call `onDrawerClose`. Passing props into a hook is valid and common when hook logic depends on prop data or prop callbacks.

```
// src/components/sidenav/useSideNav.tsx
import { useCallback, useMemo } from 'react';
import { useNavigate } from 'react-router-dom';
import DashboardIcon from '@mui/icons-material/Dashboard';
import ListAltIcon from '@mui/icons-material/ListAlt';
import AddBoxIcon from '@mui/icons-material/AddBox';
import type { IDrawerMenuItem, ISideNavProps } from './types';

export function useSideNav({ onDrawerClose }: ISideNavProps) {
    const navigate = useNavigate();

    const drawerMenu = useMemo<IDrawerMenuItem[]>(
        () => [
            { text: 'Dashboard', icon: <DashboardIcon />, path: '/' },
            { text: 'Products List', icon: <ListAltIcon />, path: '/products-list' },
            { text: 'Create Product', icon: <AddBoxIcon />, path: '/add-product' },
        ],
        []
    );

    const handleMenuClick = useCallback(
        (path: string) => {
            navigate(path);
            onDrawerClose();
        },
        [navigate, onDrawerClose]
    );

    return { drawerMenu, handleMenuClick };
}
```

Two important details:

1. The file is `.tsx` because it contains JSX icons.
2. Adding a new menu item is now a one-line change in one hook, not an edit buried in markup. If there were fifty menu items, the UI file would still be short.

7.5 `types.ts` and the icon type

```ts
// src/components/sidenav/types.ts
import type { JSX } from 'react';

export interface ISideNavProps {
    openDrawer: boolean;
    onDrawerClose: () => void;
}

export interface IDrawerMenuItem {
    text: string;
    icon: JSX.Element;
    path: string;
}
```

The type of `icon` deserves attention. It is **not** a string, number, or boolean; it is the result of rendering a component, i.e. a React element. In the lecture's words: `React.JSX.Element`. If it were typed as `string`, `<DashboardIcon />` would be rejected.

7.6 The component after extraction

```ts
// src/components/sidenav/SideNav.tsx
import { memo } from 'react';
import {
    Drawer,
    List,
    ListItem,
    ListItemButton,
    ListItemIcon,
    ListItemText,
} from '@mui/material';
import { useSideNav } from './useSideNav';
import type { ISideNavProps } from './types';

export const SideNav = memo(({ openDrawer, onDrawerClose }: ISideNavProps) => {
    const { drawerMenu, handleMenuClick } = useSideNav({ openDrawer, onDrawerClose });

    return (
        <Drawer open={openDrawer} onClose={onDrawerClose}>
            <List sx={{ width: 250 }}>
                {drawerMenu.map((menuItem) => (
                    <ListItem key={menuItem.text} disablePadding>
```

```
                        <ListItemButton onClick={() =>
handleMenuClick(menuItem.path)}>
                            <ListItemIcon>{menuItem.icon}</ListItemIcon>
                            <ListItemText primary={menuItem.text} />
                        </ListItemButton>
                    </ListItem>
                ))}
            </List>
        </Drawer>
    );
});
```

```
// src/components/application-topbar/ApplicationTopBar.tsx
import { memo } from 'react';
import { AppBar, IconButton, Toolbar, Typography } from '@mui/material';
import MenuIcon from '@mui/icons-material/Menu';
import type { IApplicationTopBarProps } from './types';

export const ApplicationTopBar = memo(
    ({ menuIconClickHandler }: IApplicationTopBarProps) => (
        <AppBar position="fixed">
            <Toolbar>
                <IconButton
                    color="inherit"
                    edge="start"
                    aria-label="open drawer"
                    onClick={menuIconClickHandler}
                    sx={{ mr: 2 }}
                >
                    <MenuIcon />
                </IconButton>
                <Typography variant="h6" component="div" noWrap>
                    Product Management App
                </Typography>
            </Toolbar>
        </AppBar>
    )
);
```

The top-bar folder needs only `types.ts`; there is no logic worth a hook yet. The lecture is explicit that you should not create a hook just to tick a box — create it when there is logic to move.

7.7 Errors and error boundaries (raised, not yet implemented)

A student asked whether an error in `application-topbar.tsx` affects `App.tsx`, and whether real applications have error handling. The lecture's answer: **any uncaught error in any component can crash the entire React tree** — the user sees a blank page. React's mechanism is an **error**

boundary, a component that catches errors from its descendants and renders a fallback UI instead of unmounting everything.

Background: error boundaries are implemented as class components with `static getDerivedStateFromError(error)` and/or `componentDidCatch(error, info)`; there is no hook equivalent for catching render errors. The lecture said this would be added to the application in a later session; it was not part of this morning's code. Do not claim it was implemented here.

> **Exam:** Moving logic into a hook improves organisation; it does **not** add error handling. Error boundaries are a separate concern, and they catch render/lifecycle errors in their subtree, not errors in event handlers or asynchronous callbacks.

8. Memoization with memo, useMemo and useCallback

8.1 Why optimise a working screen?

After the refactor the screen behaved correctly, but React re-renders a component whenever its parent re-renders or its state changes. Some of that work is unnecessary: a page whose props did not change does not need to rebuild its JSX. The lecture calls this **unnecessary re-rendering** and applies three tools.

Tool	What it memoises	Typical use here
`memo(Component)`	The **component**: skip re-render when props are shallow-equal	Pages, `AppLayout`, `SideNav`, `ApplicationTopBar`, `AppRouter`
`useMemo(fn, deps)`	A **computed value**: reuse the previous result until deps change	The `drawerMenu` array
`useCallback(fn, deps)`	A **function identity**: reuse the same function until deps change	`handleOpenDrawer`, `handleCloseDrawer`

8.2 `memo` on components

The lecture changed each page from a function declaration to a memoised arrow component:

```
import { memo } from 'react';

export const Dashboard = memo(() => (
    <Typography variant="h1" component="h1">
        Welcome to the product management app
```

```
    </Typography>
));
```

The same wrap was applied to `ProductsList`, `CreateProduct`, `AppLayout`, `SideNav`, `ApplicationTopBar`, and `AppRouter`.

`memo` performs a **shallow comparison** of props. If every prop is `===` to the previous render's prop, React reuses the previous render. With no props at all, a page is a good candidate: it only needs to re-render when its own state changes.

8.3 `useMemo` for the drawer menu

The menu array was being rebuilt on every render of the side nav. It is static data: it depends on no state and no props. Rebuilding it is wasted work, and each rebuild creates new object identities for every item.

```
const drawerMenu = useMemo<IDrawerMenuItem[]>(
    () => [
        { text: 'Dashboard', icon: <DashboardIcon />, path: '/' },
        { text: 'Products List', icon: <ListAltIcon />, path: '/products-list' },
        { text: 'Create Product', icon: <AddBoxIcon />, path: '/add-product' },
    ],
    [] // no dependencies: compute once, cache forever
);
```

With an empty dependency array, React creates the array on the first render, caches it, and returns the same array on every later render. The lecture describes it as "one-time use data, not dependent on any state — create once and memorise".

8.4 `useCallback` for handler props

This is the subtle part. `memo` on a child only helps if the child's props are stable. If the parent defines a handler inline:

```
// Every parent render creates a NEW function object
<ApplicationTopBar menuIconClickHandler={() => setOpenDrawer(true)} />
```

then `menuIconClickHandler` is a different reference on every render, the shallow comparison fails, and `memo` lets the child re-render anyway. Function definitions are re-created on each render; their *identity* changes even when their behaviour does not.

`useCallback` preserves the identity:

```
const handleOpenDrawer = useCallback(() => {
    setOpenDrawer(true);
}, []);

const handleCloseDrawer = useCallback(() => {
    setOpenDrawer(false);
}, []);
```

The dependency array is empty because these functions only call a state setter, and React guarantees that setters are stable. If a handler read a state variable or prop, that value would have to be listed in the dependency array; otherwise the callback would capture a stale value.

```
Without useCallback                    With useCallback
─────────────────────                  ─────────────────
render 1: fn#A  ──► child renders      render 1: fn#A ──► child renders
render 2: fn#B  ──► props differ       render 2: fn#A ──► props equal
                    child re-renders                      child skipped
```

8.5 What memoisation does not fix

- It does not make the first render faster; it only avoids later renders.
- It does not help if props genuinely change every time (for example, a new object literal passed inline without `useMemo`).
- It adds a comparison cost. For a tiny component rendered once, it is unnecessary.
- It must not be used to hide a state-ownership bug. The re-render is a symptom; lift state correctly first, then optimise.

> **Exam:** `memo` memoises a component, `useMemo` memoises a value, `useCallback` memoises a function. `useCallback(fn, deps)` is essentially `useMemo(() => fn, deps)`.

9. Building the Product Table

9.1 Choosing the MUI table

The product list must display data from the database. MUI offers a complex **Data Grid** under MUI X, but the lecture deliberately avoids it for now: it is a bigger topic. Instead, the standard **Table** component is used.

The product table in MySQL (`productDB` → `product`) has these columns:

Column	Type	Used in the table?
`id`	number	Not displayed; used as React `key`
`name`	string	Yes, first column
`description`	string	Yes, second column
`price`	number	Yes, third column
`quantity`	number	Yes, fourth column

9.2 Replacing the heading with an Alert

Instead of a plain `<Typography>` heading, the page uses an MUI `Alert`. The default alert icon was not wanted, so `icon={false}` removes it:

```
<Alert severity="info" icon={false}>
    List of products
</Alert>
```

Everything is wrapped in a `Box` so the alert and the table share one container, and the table gets breathing room with `mt: 2` (MUI spacing 2 = 16px; the lecture mentions 20px as an equivalent choice).

9.3 Table structure

The MUI table mirrors real HTML table structure:

```
TableContainer   (scrolling / responsive wrapper)
└── Paper        (surface / elevation)
    └── Table
        ├── TableHead
        │   └── TableRow
        │       └── TableCell × columns      ← header cells
        └── TableBody
            └── TableRow × products          ← one row per database record
                └── TableCell × columns
```

The body is dynamic: `products.map( ... )` turns each record into a `TableRow`, and `product.id` is used as the `key` because it is unique and stable. Column widths can be controlled with the `sx` prop on each `TableCell` (for example `sx={{ width: 200 }}`), and the `align` prop changes text alignment; the lecture removed the default alignments for this table.

```
// src/pages/products-list/ProductsList.tsx
import { memo } from 'react';
import {
```

```
    Alert,
    Box,
    Paper,
    Table,
    TableBody,
    TableCell,
    TableContainer,
    TableHead,
    TableRow,
} from '@mui/material';
import { useProductsList } from './useProductsList';

export const ProductsList = memo(() => {
    const { products } = useProductsList();

    return (
        <Box>
            <Alert severity="info" icon={false}>
                List of products
            </Alert>

            <TableContainer component={Paper} sx={{ mt: 2 }}>
                <Table>
                    <TableHead>
                        <TableRow>
                            <TableCell sx={{ fontWeight: 700 }}>Name</TableCell>
                            <TableCell sx={{ fontWeight: 700
}}>Description</TableCell>
                            <TableCell sx={{ fontWeight: 700 }}>Price</TableCell>
                            <TableCell sx={{ fontWeight: 700 }}>Quantity</TableCell>
                        </TableRow>
                    </TableHead>
                    <TableBody>
                        {products.map((product) => (
                            <TableRow key={product.id}>
                                <TableCell>{product.name}</TableCell>
                                <TableCell>{product.description}</TableCell>
                                <TableCell>{product.price}</TableCell>
                                <TableCell>{product.quantity}</TableCell>
                            </TableRow>
                        ))}
                    </TableBody>
                </Table>
            </TableContainer>
        </Box>
    );
});
```

9.4 The open question: why the headers were not bold

The lecture wrapped the headers in `TableHead` but noticed the column titles were still not bold, even though the MUI documentation example appeared bold. The reason was not resolved before the session ended; the lecturer explicitly deferred it ("we will see this later").

Background: MUI's `TableCell` renders a `<td>` by default, and a `<td>` has no bold styling. The documentation example often depends on theme typography or a `component="th"` header cell. Two reliable fixes are:

```
// Fix 1 — explicit style on each header cell
<TableCell sx={{ fontWeight: 700 }}>Name</TableCell>

// Fix 2 — semantic header cell (th is bold by default)
<TableCell component="th" scope="col">Name</TableCell>
```

Using `component="th" scope="col"` is also better for accessibility, because screen readers can then associate the header with its column.

> **Pitfall:** A table that *looks* like a table is not necessarily accessible. Header cells should be real `<th>` elements (or styled explicitly), and the body rows need a stable `key`. Using the array index as the key breaks when rows are inserted or deleted.

10. Fetching Products from Our Backend

10.1 The backend is the one built earlier

The APIs from the 29 October session ("CRUD APIs") were copied into the day's folder and installed with `npm i`. The Express server runs on **port 7500**, and the relevant endpoint is:

Method	URL	Meaning
GET	`/product`	Return every product
POST	`/product`	Add a product
PUT	`/product/:id`	Update a product
DELETE	`/product/:id`	Delete a product

The lecture points out that until now the frontend called **dummy APIs** such as JSON Placeholder. This is the first time the React app calls its **own** backend.

One temporary change was made: the product router's `verifyToken` middleware was removed, so the frontend could call the API without sending a JWT. The lecture is clear that this is temporary: when token authentication is integrated on the frontend, the middleware must be restored.

> **Pitfall:** Commenting out `verifyToken` leaves the API open. It is acceptable for a classroom integration test, never for code you ship. Re-enable it as soon as the frontend starts sending the `Authorization` header.

10.2 Type safety for the response

Without a type, `useState([])` infers `never[]` (or, with a generic, `any[]`), and the compiler cannot warn about `product.nmae`. A `types.ts` file describes the **shape** of the server response:

```ts
// src/pages/products-list/types.ts
export interface IProduct {
    id: number;
    name: string;
    description: string;
    price: number;
    quantity: number;
}
```

The state is then declared as an **array of products**, not a single product:

```ts
const [products, setProducts] = useState<IProduct[]>([]);
```

`IProduct[]` means "an array whose elements are `IProduct`". Assigning a string, an object of the wrong shape, or an array of something else is now a compile-time error.

10.3 `useEffect`: run once when the component loads

A fetch is a side effect, so it belongs in `useEffect`. The lecture wants the request sent **as soon as the component loads**, equivalent to the class lifecycle method `componentDidMount`. That is expressed by an empty dependency array:

```ts
useEffect(() => {
    fetchProductsData();
}, []);
```

`useEffect` cannot receive an `async` function directly, because an async function returns a Promise and an effect must return either nothing or a cleanup function. The workaround is to define an async function outside the effect and call it inside.

10.4 The first version, using `fetch`

```
import { useCallback, useEffect, useState } from 'react';
import type { IProduct } from './types';

export function useProductsList() {
    const [products, setProducts] = useState<IProduct[]>([]);

    const fetchProductsData = useCallback(async () => {
        try {
            const response = await fetch('http://127.0.0.1:7500/product');
            const productsData: IProduct[] = await response.json();
            setProducts(productsData);
        } catch (error) {
            console.error('Could not fetch products:', error);
        }
    }, []);

    useEffect(() => {
        fetchProductsData();
    }, [fetchProductsData]);

    return { products };
}
```

Step by step:

1. `fetch(url)` sends the GET request and returns a **Promise**.
2. `await` unwraps the Promise into a `Response` object.
3. `response.json()` reads the body and parses it; it **also returns a Promise**, so it needs a second `await`.
4. The parsed array is stored with `setProducts`.
5. A rejected Promise (network failure, server down) is caught and logged.

The lecture initially wrapped the fetch in `useCallback` and used an empty dependency array. That produces an ESLint warning (`react-hooks/exhaustive-deps`) because the effect uses `fetchProductsData` but does not list it. The lecture's quick fix was a file-level disable comment:

```
/* eslint-disable react-hooks/exhaustive-deps */
```

That silences the warning, but the cleaner fix is the one shown above: list `fetchProductsData` in the effect's dependency array. Because the callback itself has stable dependencies (`[]`), the effect still runs only once.

> **Exam:** An empty dependency array means "run after the first render only" (in development with `StrictMode`, React intentionally runs it twice to expose missing cleanups). It does not mean "never run again" in the sense of disabling the effect; it means "no dependency can trigger it again".

10.5 Rendering the fetched rows

The `ProductsList` component destructures `products` from the hook and maps it. On first render the array is empty, so `TableBody` is empty. When the Promise resolves, state changes, the component re-renders, and rows appear.

```
const { products } = useProductsList();
// ...
{products.map((product) => (
    <TableRow key={product.id}> ... </TableRow>
))}
```

At this point the lecture ran the app and saw **no data** and a console error. That error is the subject of the next section.

11. CORS: Why the Request Was Blocked

11.1 Reading the error

The browser console showed something equivalent to:

```
Access to fetch at 'http://127.0.0.1:7500/product' from origin
'http://localhost:5173' has been blocked by CORS policy:
No 'Access-Control-Allow-Origin' header is present on the requested resource.
```

Decoded:

- The frontend runs at `http://localhost:5173` (Vite's dev server).
- The backend runs at `http://127.0.0.1:7500` (Express).
- The browser sent the request from the first origin to the second.
- The **backend's response was rejected by the browser** because it did not carry the header that says "this origin is allowed".

The lecture's summary: *the request we are making from our own frontend has been blocked at the backend.* Note the precise mechanics: the server may have processed the request, but the **browser**

refuses to expose the response to JavaScript when the CORS check fails. CORS is enforced by the browser, configured by the server.

11.2 What an "origin" is

Background: An origin is the triple **scheme + host + port**. `http://localhost:5173` and `http://127.0.0.1:7500` differ in both host and port, so they are different origins even though both run on the same machine. A same-origin request (same scheme, host, and port) is not subject to the CORS check; a cross-origin request is. This is why the lecture's observation that "the origin is the same" is easy to make but not quite correct: the port is part of the origin. The practical conclusion is unchanged — the request was cross-origin and the backend had no CORS configuration.

Some sources also explain CORS with two frontends such as `192.168.12.1` and `192.168.13.2`; those are simply two different origins, and the backend decides which of them may read its responses.

11.3 Why the block exists

Imagine an enterprise backend with APIs for orders, products, and payments. Should **any** website be able to write JavaScript that calls those APIs with the user's cookies? Obviously not. The API owner wants only its own clients — its website, its Android app, its iOS app — to be able to call it.

CORS (**Cross-Origin Resource Sharing**) is the browser-side security policy that makes the server declare which origins may read its resources. Without a declaration, the safe default is **deny**. The lecture's analogy: any random frontend must not be able to call your APIs; only the frontend you deliberately allow.

Question	Answer
Who sends the request?	The browser, on behalf of the frontend
Who decides whether the origin is allowed?	The backend, via CORS response headers
Where is CORS configured?	Backend only, not React
What happens with no configuration?	Cross-origin requests are blocked
Does the same apply in Java, Python, PHP, .NET?	Yes — every backend technology has CORS configuration

11.4 The fix: enable CORS on Express

```
npm install cors
```

```
// index.js
const express = require('express');
const cors = require('cors');

const app = express();

// Development shortcut: allow every origin.
// Nothing will be blocked.
app.use(cors());

app.use(express.json());
```

`cors()` returns an Express **middleware**. Middleware runs before the route handler, so it can inspect the request's `Origin` and attach the appropriate `Access-Control-Allow-Origin` response header. The lecture compares it to `express.json()`: both are functions that return middleware for `app.use`.

After adding the middleware, the server must be **restarted** (`node index.js`). Node does not hot-reload a plain Express server. Once restarted, the same GET request succeeded and the table filled with the database rows.

11.5 Restricting origins in production

`app.use(cors())` with no options allows **all** origins. That is fine for a classroom, but it defeats the purpose in production. The `cors` package accepts an options object:

```
// Allow only the frontends you control
app.use(
    cors({
        origin: ['http://localhost:5173', 'https://admin.example.com'],
    })
);
```

Requests from any other origin receive a CORS error. The lecture also shows the single-origin form `cors({ origin: 'https://example.com' })`.

A student asked whether hosting the frontend and backend on the same server would avoid the problem. The lecture's point is that deployment alone is not a fix: if the backend never sends the CORS header, the browser still blocks the cross-origin response. The configuration must be added to whichever backend is deployed, in any language.

> **Pitfall:** CORS errors cannot be fixed in React. Adding headers in `fetch`, changing the component, or installing a frontend package will not help. The allow-list belongs on the server. Also, do not "fix" production by leaving `cors()` wide open.

11.6 What CORS does not do

CORS is not authentication. It stops a *browser page* from reading responses from another origin; it does not stop a server-side script, `curl`, or a mobile app from calling the API. That is why the product API still needs JWT authentication, and why `verifyToken` must be restored.

> **Exam:** CORS = Cross-Origin Resource Sharing. It is a browser-enforced policy, configured on the backend, that decides which origins may read the backend's responses. `npm i cors` + `app.use(cors())` enables it in Express; `cors({ origin: [...] })` restricts it.

12. Axios and a Central API Client

12.1 Problems with raw `fetch`

The table loaded after the CORS fix, but the lecture used the moment to compare `fetch` with a dedicated HTTP client. `fetch` is workable for a simple GET, but:

1. **POST/PUT/DELETE need a configuration object.** You must pass `method`, `headers`, and `body` manually.
2. **The body must be stringified.** `body: JSON.stringify(formData)`; a plain object is not accepted.
3. **The full URL is repeated everywhere.** Every component that calls the API writes `http://127.0.0.1:7500/...` again.
4. **No central configuration.** Base URL, timeout, auth headers, and common error handling must be re-implemented or copied.
5. **No interceptors.** Cross-cutting concerns (attach token, refresh token, show a global error) have no built-in hook.

The lecture's analogy: `fetch` is a **Maruti Alto**; Axios is a **Fortuner**. Both get you there, but one provides far more capability out of the box.

12.2 Installing and using Axios

```
npm i axios
```

```
import axios from 'axios';

const response = await axios.get('http://127.0.0.1:7500/product');
const productsData: IProduct[] = response.data;
setProducts(productsData);
```

Differences from `fetch`:

Concern	`fetch`	`axios`
GET call	`fetch(url)`	`axios.get(url)`
Read JSON	`await response.json()` (extra Promise)	`response.data` (already parsed)
POST body	`body: JSON.stringify(data)` + headers	`axios.post(url, data)`
PUT / DELETE	manual `method` option	`axios.put( ... )`, `axios.delete( ... )`
Response object	`Response` (status, headers, body stream)	object with `data`, `status`, `headers`
Base URL	repeated in every call	configurable once with `axios.create`
Timeout	not built in	`timeout` option
Interceptors	none	built in

Axios still returns a **Promise**, so `await` and `try/catch` work exactly as before. The difference is what the resolved value contains: `response.data` is the parsed body, while `response.status` and `response.headers` are also available.

12.3 Moving the fetch into `useProductsList`

The logic was moved out of the component into a hook, following section 7:

```
// src/pages/products-list/useProductsList.ts
import { useCallback, useEffect, useState } from 'react';
import apiClient from '../../api-client/api-client';
import type { IProduct } from './types';

export function useProductsList() {
    const [products, setProducts] = useState<IProduct[]>([]);

    const fetchProductsData = useCallback(async () => {
        try {
            const response = await apiClient.get<IProduct[]>('/product');
            setProducts(response.data);
        } catch (error) {
```

```
            console.error('Could not fetch products:', error);
        }
    }, []);

    useEffect(() => {
        fetchProductsData();
    }, [fetchProductsData]);

    return { products };
}
```

The component now contains only UI:

```
const { products } = useProductsList();
```

12.4 A single configured client: `axios.create`

Hard-coding the base URL in every hook still repeats knowledge. Axios can create a configured **instance** once:

```
// src/api-client/api-client.ts
import axios from 'axios';

const apiClient = axios.create({
    baseURL: 'http://127.0.0.1:7500',
    timeout: 5000,
    headers: {
        'Content-Type': 'application/json',
    },
});

export default apiClient;
```

Then every call supplies only the path:

```
const response = await apiClient.get<IProduct[]>('/product');
// later: apiClient.post('/product', newProduct)
// later: apiClient.put(`/product/${id}`, updatedProduct)
// later: apiClient.delete(`/product/${id}`)
```

What the configuration gives you:

Option	Effect
`baseURL`	Prepended to every relative path; change the server once, not in fifty files

Option	Effect
`timeout: 5000`	Cancel a request that takes longer than 5 seconds and throw an error instead of waiting forever
`headers`	Default headers applied to every request; JSON content type and (later) `Authorization`

The timeout is not cosmetic. Without it, a request to a dead server can hang for a long time. With it, the Promise rejects after five seconds, the `catch` block runs, and the UI can show "server timed out, please try again".

Background: the same instance can also carry **interceptors** — functions that run before a request (attach the JWT) or after a response (handle 401 by redirecting to login). The lecture mentions interceptors as a reason Axios is more powerful than `fetch`; they were not implemented in this session.

> **Exam:** `axios.create({ baseURL, timeout, headers })` returns a reusable client instance. Call `apiClient.get('/product')` instead of repeating the full URL. Use `response.data`; do not call `response.json()`.

13. Combined Worked Example

This section ties the whole morning together into one runnable frontend. It assumes the Day 8 backend is running on port 7500 with `GET /product` returning an array of product objects. The MUI markup around the parts the lecture showed (drawer list, app bar, page placeholders) is reconstructed so the example compiles as a whole; the state ownership, prop flow, routing, hooks, memoisation, table mapping, and API calls follow the session exactly.

13.1 Project setup

```
npm create vite@latest crud-ui -- --template react-ts
cd crud-ui
npm install
npm install @mui/material @emotion/react @emotion/styled @mui/icons-material
npm install react-router-dom axios
npm run dev
```

The frontend runs on `http://localhost:5173`; the backend runs on `http://127.0.0.1:7500`. They are different origins — which is exactly why the backend needs CORS.

13.2 File tree

```
src/
├── main.tsx
├── index.css
├── App.tsx
├── api-client/
│   └── api-client.ts
├── components/
│   ├── app-layout/
│   │   ├── AppLayout.tsx
│   │   └── useAppLayout.ts
│   ├── app-router/
│   │   └── AppRouter.tsx
│   ├── application-topbar/
│   │   ├── ApplicationTopBar.tsx
│   │   └── types.ts
│   └── sidenav/
│       ├── SideNav.tsx
│       ├── useSideNav.tsx
│       └── types.ts
└── pages/
    ├── dashboard/
    │   └── Dashboard.tsx
    ├── products-list/
    │   ├── ProductsList.tsx
    │   ├── useProductsList.ts
    │   └── types.ts
    └── create-product/
        └── CreateProduct.tsx
```

13.3 Entry point and app shell

```tsx
// src/main.tsx
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import App from './App';
import './index.css';

createRoot(document.getElementById('root')!).render(
    <StrictMode>
        <App />
    </StrictMode>
);
```

```css
/* src/index.css — minimal reset so the fixed AppBar sits flush */
body {
    margin: 0;
    font-family: "Segoe UI", Arial, sans-serif;
}
```

```tsx
// src/App.tsx
import { BrowserRouter } from 'react-router-dom';
import { AppRouter } from './components/app-router/AppRouter';

function App() {
    return (
        <BrowserRouter>
            <AppRouter />
        </BrowserRouter>
    );
}

export default App;
```

13.4 Routing

```tsx
// src/components/app-router/AppRouter.tsx
import { memo } from 'react';
import { Route, Routes } from 'react-router-dom';
import { AppLayout } from '../app-layout/AppLayout';
import { Dashboard } from '../../pages/dashboard/Dashboard';
import { ProductsList } from '../../pages/products-list/ProductsList';
import { CreateProduct } from '../../pages/create-product/CreateProduct';

export const AppRouter = memo(() => (
    <Routes>
        <Route path="/" element={<AppLayout />}>
            <Route index element={<Dashboard />} />
            <Route path="products-list" element={<ProductsList />} />
            <Route path="add-product" element={<CreateProduct />} />
        </Route>
    </Routes>
));
```

13.5 Layout and its hook

```tsx
// src/components/app-layout/AppLayout.tsx
import { memo } from 'react';
import { Box, Toolbar } from '@mui/material';
import { Outlet } from 'react-router-dom';
import { ApplicationTopBar } from '../application-topbar/ApplicationTopBar';
import { SideNav } from '../sidenav/SideNav';
import { useAppLayout } from './useAppLayout';

export const AppLayout = memo(() => {
    const { openDrawer, handleOpenDrawer, handleCloseDrawer } = useAppLayout();

    return (
```

```
        <>
            <ApplicationTopBar menuIconClickHandler={handleOpenDrawer} />
            <SideNav openDrawer={openDrawer} onDrawerClose={handleCloseDrawer} />
            <Box component="main" sx={{ flexGrow: 1, p: 4 }}>
                <Toolbar />
                <Outlet />
            </Box>
        </>
    );
});
```

```
// src/components/app-layout/useAppLayout.ts
import { useCallback, useState } from 'react';

export function useAppLayout() {
    const [openDrawer, setOpenDrawer] = useState(false);

    const handleOpenDrawer = useCallback(() => setOpenDrawer(true), []);
    const handleCloseDrawer = useCallback(() => setOpenDrawer(false), []);

    return {
        openDrawer,
        setOpenDrawer,
        handleOpenDrawer,
        handleCloseDrawer,
    };
}
```

13.6 Top bar

```
// src/components/application-topbar/types.ts
export interface IApplicationTopBarProps {
    menuIconClickHandler: () => void;
}
```

```
// src/components/application-topbar/ApplicationTopBar.tsx
import { memo } from 'react';
import { AppBar, IconButton, Toolbar, Typography } from '@mui/material';
import MenuIcon from '@mui/icons-material/Menu';
import type { IApplicationTopBarProps } from './types';

export const ApplicationTopBar = memo(
    ({ menuIconClickHandler }: IApplicationTopBarProps) => (
        <AppBar position="fixed">
            <Toolbar>
                <IconButton
                    color="inherit"
```

```
                    edge="start"
                    aria-label="open drawer"
                    onClick={menuIconClickHandler}
                    sx={{ mr: 2 }}
                >
                    <MenuIcon />
                </IconButton>
                <Typography variant="h6" component="div" noWrap>
                    Product Management App
                </Typography>
            </Toolbar>
        </AppBar>
    )
);
```

13.7 Side nav

```
// src/components/sidenav/types.ts
import type { JSX } from 'react';

export interface ISideNavProps {
    openDrawer: boolean;
    onDrawerClose: () => void;
}

export interface IDrawerMenuItem {
    text: string;
    icon: JSX.Element;
    path: string;
}
```

```
// src/components/sidenav/useSideNav.tsx
import { useCallback, useMemo } from 'react';
import { useNavigate } from 'react-router-dom';
import DashboardIcon from '@mui/icons-material/Dashboard';
import ListAltIcon from '@mui/icons-material/ListAlt';
import AddBoxIcon from '@mui/icons-material/AddBox';
import type { IDrawerMenuItem, ISideNavProps } from './types';

export function useSideNav({ onDrawerClose }: ISideNavProps) {
    const navigate = useNavigate();

    const drawerMenu = useMemo<IDrawerMenuItem[]>(
        () => [
            { text: 'Dashboard', icon: <DashboardIcon />, path: '/' },
            { text: 'Products List', icon: <ListAltIcon />, path: '/products-list' },
            { text: 'Create Product', icon: <AddBoxIcon />, path: '/add-product' },
        ],
```

```
        []
    );

    const handleMenuClick = useCallback(
        (path: string) => {
            navigate(path);
            onDrawerClose();
        },
        [navigate, onDrawerClose]
    );

    return { drawerMenu, handleMenuClick };
}
```

```
// src/components/sidenav/SideNav.tsx
import { memo } from 'react';
import {
    Drawer,
    List,
    ListItem,
    ListItemButton,
    ListItemIcon,
    ListItemText,
} from '@mui/material';
import { useSideNav } from './useSideNav';
import type { ISideNavProps } from './types';

export const SideNav = memo(({ openDrawer, onDrawerClose }: ISideNavProps) => {
    const { drawerMenu, handleMenuClick } = useSideNav({ openDrawer, onDrawerClose });

    return (
        <Drawer open={openDrawer} onClose={onDrawerClose}>
            <List sx={{ width: 250 }}>
                {drawerMenu.map((menuItem) => (
                    <ListItem key={menuItem.text} disablePadding>
                        <ListItemButton onClick={() =>
handleMenuClick(menuItem.path)}>
                            <ListItemIcon>{menuItem.icon}</ListItemIcon>
                            <ListItemText primary={menuItem.text} />
                        </ListItemButton>
                    </ListItem>
                ))}
            </List>
        </Drawer>
    );
});
```

13.8 Pages

```tsx
// src/pages/dashboard/Dashboard.tsx
import { memo } from 'react';
import { Typography } from '@mui/material';

export const Dashboard = memo(() => (
    <Typography variant="h1" component="h1">
        Welcome to the product management app
    </Typography>
));
```

```tsx
// src/pages/create-product/CreateProduct.tsx
import { memo } from 'react';
import { Typography } from '@mui/material';

export const CreateProduct = memo(() => (
    <Typography variant="h1" component="h1">
        Add a new product
    </Typography>
));
```

```tsx
// src/pages/products-list/types.ts
export interface IProduct {
    id: number;
    name: string;
    description: string;
    price: number;
    quantity: number;
}
```

```tsx
// src/pages/products-list/useProductsList.ts
import { useCallback, useEffect, useState } from 'react';
import apiClient from '../../api-client/api-client';
import type { IProduct } from './types';

export function useProductsList() {
    const [products, setProducts] = useState<IProduct[]>([]);

    const fetchProductsData = useCallback(async () => {
        try {
            const response = await apiClient.get<IProduct[]>('/product');
            setProducts(response.data);
        } catch (error) {
            console.error('Could not fetch products:', error);
        }
    }, []);
```

```
    useEffect(() => {
        fetchProductsData();
    }, [fetchProductsData]);

    return { products };
}
```

```
// src/pages/products-list/ProductsList.tsx
import { memo } from 'react';
import {
    Alert,
    Box,
    Paper,
    Table,
    TableBody,
    TableCell,
    TableContainer,
    TableHead,
    TableRow,
} from '@mui/material';
import { useProductsList } from './useProductsList';

export const ProductsList = memo(() => {
    const { products } = useProductsList();

    return (
        <Box>
            <Alert severity="info" icon={false}>
                List of products
            </Alert>

            <TableContainer component={Paper} sx={{ mt: 2 }}>
                <Table>
                    <TableHead>
                        <TableRow>
                            <TableCell sx={{ fontWeight: 700 }}>Name</TableCell>
                            <TableCell sx={{ fontWeight: 700
}}>Description</TableCell>
                            <TableCell sx={{ fontWeight: 700 }}>Price</TableCell>
                            <TableCell sx={{ fontWeight: 700 }}>Quantity</TableCell>
                        </TableRow>
                    </TableHead>
                    <TableBody>
                        {products.map((product) => (
                            <TableRow key={product.id}>
                                <TableCell>{product.name}</TableCell>
                                <TableCell>{product.description}</TableCell>
                                <TableCell>{product.price}</TableCell>
                                <TableCell>{product.quantity}</TableCell>
```

```
                            </TableRow>
                        ))}
                    </TableBody>
                </Table>
            </TableContainer>
        </Box>
    );
});
```

13.9 API client

```
// src/api-client/api-client.ts
import axios from 'axios';

const apiClient = axios.create({
    baseURL: 'http://127.0.0.1:7500',
    timeout: 5000,
    headers: {
        'Content-Type': 'application/json',
    },
});

export default apiClient;
```

13.10 Backend CORS configuration

The complete product CRUD backend from Day 8/9 is not reproduced here; this is the part that changed in this session. `verifyToken` was temporarily removed from the product router so the frontend could call the API without a JWT. It must be restored when authentication is wired up.

```
// index.js — CORS-enabled Express entry point
const express = require('express');
const cors = require('cors');
const productRouter = require('./routes/productRouter');

const app = express();

// Development: allow every origin.
// Production: replace with cors({ origin: ['https://your-frontend.example.com'] })
app.use(cors());
app.use(express.json());

app.use('/product', productRouter);

app.listen(7500, () => {
    console.log('CRUD API listening on http://127.0.0.1:7500');
});
```

13.11 End-to-end trace

Follow one click from the UI to the rendered row:

1. The browser opens `http://localhost:5173/`.
2. `BrowserRouter` reads the URL; `AppRouter` matches the parent `path="/"`.
3. `AppLayout` renders first. `useAppLayout` initialises `openDrawer = false`.
4. `ApplicationTopBar` receives `handleOpenDrawer`; `SideNav` receives `openDrawer` and `handleCloseDrawer`. `Outlet` is empty until a child route matches.
5. The `index` child route matches, so `Dashboard` appears at the `Outlet` position.
6. The user clicks the menu icon → `handleOpenDrawer()` → `openDrawer = true` → MUI `Drawer` slides in. The state lives in `AppLayout`, not in `SideNav`.
7. The user clicks "Products List" → `useSideNav` calls `navigate('/products-list')`, then `onDrawerClose()` → the drawer closes.
8. The router matches the child route `products-list`; `Outlet` now renders `ProductsList`.
9. `useProductsList` runs its effect once, calling `apiClient.get('/product')`.
10. Axios prepends the base URL, producing `http://127.0.0.1:7500/product`, and sends the request with an `Origin: http://localhost:5173` header.
11. The Express `cors()` middleware sees the request before the route handler and attaches `Access-Control-Allow-Origin` to the response.
12. The browser accepts the response; Axios resolves with `response.data`, the parsed array.
13. `setProducts(array)` schedules a re-render. `ProductsList` maps the array into `TableRow`s, each keyed by `product.id`.
14. The table shows the database rows. If the request had failed, the `catch` block would log the error and the table would remain empty.

14. Quick Revision Checklist

Topic	One-line takeaway
Sibling data sharing	Move shared state to the closest common parent; pass values down and callbacks up
Lifting state up	`AppLayout` owns `openDrawer` and the open/close handlers; both children receive props
Fragment wrapper	`<> … </>` is a container with no DOM node; prefer it over a stray `div` / `Box`

Topic	One-line takeaway
Typed props	`menuIconClickHandler: () ⇒ void`, `openDrawer: boolean`, `onDrawerClose: () ⇒ void`
Pages vs components	Screens live in `src/pages/`; reusable and routing components live in `src/components/`
Nested routes	Parent `<Route path="/" element={<AppLayout />}>` wraps child routes; layout is declared once
`Outlet`	Placeholder that renders the child route matched for the current URL
Side-nav navigation	`useNavigate()` + a `path` field on each menu item; call `onDrawerClose()` after navigating
`AppRouter`	Route table extracted out of `App.tsx`; `App` only renders `BrowserRouter` + `AppRouter`
Custom hooks	Business logic (state + handlers) moves to `use<Component>.ts(x)`; UI stays in the `.tsx`
Folder-per-component	`<Component>.tsx`, `use<Component>.ts`, `types.ts` in one folder
`memo`	Skips a component re-render when its props are shallow-equal
`useMemo`	Caches a computed value, e.g. the static `drawerMenu` array, with `[]` deps
`useCallback`	Keeps a function's identity stable so `memo` children can actually skip renders
MUI table	`TableContainer > Paper > Table > TableHead/TableBody > TableRow > TableCell`
List keys	Use the database `id`, not the array index
`useEffect` fetch	Empty `[]` deps ≈ `componentDidMount`; async work is wrapped in an inner function
Response typing	`IProduct[]` describes the array shape and enables compiler checks
CORS error	Browser blocks a cross-origin response unless the backend allows the origin
CORS fix	Backend: `npm i cors`, `app.use(cors())` in dev, `cors({ origin: [ ... ] })` in production
Axios	`axios.get/post/put/delete`; JSON is parsed at `response.data`, not `response.json()`

Topic	One-line takeaway
API client	`axios.create({ baseURL, timeout, headers })` centralises configuration
Token auth	`verifyToken` was removed only for the integration test; restore it afterwards

15. Practice Exercises

Concept

1. Why can `SideNav` not read `ApplicationTopBar`'s props directly, and what is the general rule for fixing it?
2. What exactly does `<Outlet />` render, and why can `AppLayout` not hard-code `<Dashboard />` in its place?
3. Explain the difference between `memo`, `useMemo`, and `useCallback`, and give one use of each from this session.

Application

4. Add a new page `/settings` with a `Settings` component and a "Settings" drawer item. List every file you must touch and what changes in each.
5. `AppLayout` has three pieces of shared data: `openDrawer`, `handleOpenDrawer`, and `handleCloseDrawer`. State which child receives which, and why the top bar does not need the state.
6. You need to send a POST request to `/product` with a JavaScript object `newProduct`. Write the Axios call and the equivalent `fetch` call, and name two advantages of the Axios version.

Predict the output

7. The parent route is `<Route path="/" element={<AppLayout />}>` with children `index`, `products-list`, and `add-product`. The URL is `/products-list`. Which components render, and in what nesting order?
8. A menu item's click handler calls only `navigate(menuItem.path)` and not `onDrawerClose()`. The page changes successfully. What does the user see on screen, and why?
9. The backend has CORS enabled but the frontend calls `http://127.0.0.1:7500/product` from `http://localhost:5173` with `fetch`. A colleague says "the origins are the same because both are localhost". Is the request cross-origin? Will the browser enforce CORS?

Spot the bug

10. The following effect is added to a component. Explain the error and give the corrected version.

```
useEffect(async () => {
    const response = await fetch('http://127.0.0.1:7500/product');
    const data = await response.json();
    setProducts(data);
}, []);
```

11. The drawer menu item interface is declared without `path`, but each object includes `path: '/'`. What does TypeScript report, and what is the fix?

```
export interface IDrawerMenuItem {
    text: string;
    icon: JSX.Element;
}
```

12. A developer wraps `ProductsList` in `memo` but passes the page an inline callback:

```
<ProductsList onLoaded={() => console.log('loaded')} />
```

The parent re-renders often, and `ProductsList` still re-renders every time. Why did `memo` not help, and what should be done?

16. Practice Answers

1. `SideNav` and `ApplicationTopBar` are siblings, and React props only flow from parent to child. There is no sibling-to-sibling channel. The fix is to lift the shared state and handlers into their closest common parent (`AppLayout`) and pass them down as props.

2. `Outlet` renders the child route that the router matched for the current URL. `AppLayout` cannot hard-code `<Dashboard />` because the correct page depends on the browser URL at runtime; the same layout serves `/` , `/products-list` , `/add-product` , and any future page.

3. `memo(Component)` memoises a component and skips re-rendering when props are shallow-equal (used on `Dashboard` , `ProductsList` , `AppLayout` , `SideNav` , `ApplicationTopBar` , `AppRouter`). `useMemo(fn, deps)` memoises a computed value (used for `drawerMenu`). `useCallback(fn, deps)` memoises a function's identity (used for `handleOpenDrawer` and `handleCloseDrawer`).

4. Create `src/pages/settings/Settings.tsx` (a memoised component). In `src/components/app-router/AppRouter.tsx` , import it and add `<Route path="settings" element={<Settings />} />` inside the `AppLayout` parent route. In `src/components/sidenav/useSideNav.tsx` , import a `SettingsIcon` and add `{ text:`

`'Settings', icon: <SettingsIcon />, path: '/settings' }` to `drawerMenu`. No change is needed in `App.tsx` or `AppLayout.tsx`, because the layout and the route table are already generic.

5. `ApplicationTopBar` receives `handleOpenDrawer` as `menuIconClickHandler`. `SideNav` receives `openDrawer` (to control the MUI `Drawer`) and `handleCloseDrawer` as `onDrawerClose`. The top bar does not need `openDrawer` because it never displays the drawer; it only triggers the open action.

6. Axios:

```
const response = await apiClient.post('/product', newProduct);
console.log(response.data);
```

`fetch`:

```
const response = await fetch('http://127.0.0.1:7500/product', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(newProduct),
});
const data = await response.json();
```

Advantages: the object is sent directly without `JSON.stringify`, and the base URL plus default headers come from the configured `apiClient` instead of being repeated. Axios also gives `response.data` without a second parsing Promise.

7. `AppLayout` renders first, containing `ApplicationTopBar`, `SideNav`, and the main `Box`. The router matches the `products-list` child, so `ProductsList` renders at the `Outlet` position inside that `Box`. Nesting order: `BrowserRouter > AppRouter > Routes > AppLayout > (ApplicationTopBar, SideNav, Box > Outlet > ProductsList)`.

8. The product list page appears, but the drawer remains open because `openDrawer` is still `true` in `AppLayout`. Navigation changes the URL; it does not change drawer state. The handler must also call `onDrawerClose()` after `navigate( ... )`.

9. The request is cross-origin. An origin is scheme + host + port; `localhost:5173` and `127.0.0.1:7500` differ in both host and port. If the backend allows that origin, the browser accepts the response; without the CORS header, it blocks it. (If the request were truly same-origin, CORS would not be enforced at all.)

10. `useEffect` must return either nothing or a cleanup function, but an `async` callback returns a Promise. React cannot use that as a cleanup, and the console reports the invalid effect. Wrap the async work in a function defined inside the effect:

```
useEffect(() => {
    const loadProducts = async () => {
        const response = await fetch('http://127.0.0.1:7500/product');
        const data = await response.json();
        setProducts(data);
    };

    loadProducts();
}, []);
```

For a reusable version, define `fetchProductsData` with `useCallback` outside the effect and call it inside, then list it in the dependency array.

11. TypeScript reports that `path` does not exist on type `IDrawerMenuItem` (excess property checking on the object literals, and missing property when reading `menuItem.path`). Add the field to the interface:

```
export interface IDrawerMenuItem {
    text: string;
    icon: JSX.Element;
    path: string;
}
```

12. `memo` compares props shallowly. The inline arrow function creates a **new function object on every parent render**, so the `onLoaded` prop is never `===` to the previous one and the comparison fails. Define the callback with `useCallback` (with correct dependencies) and pass the stable reference, or move the callback out of the render if it truly needs no props.